linux串口编程

1.串口通信

tx:发送数据线

rx：接收数据线

```
A(开发板串口)          B(模块)
TX<---->RX
RX<---->TX
VCC<---->VCC
GND<---->GND
```

linux中串口通信程序实现步骤：linux 一切皆文件

串口的操作

```
1.打开串口
    //6818串口所对应的文件名
    #define COM2_6818 "/dev/ttySAC1"
    #define COM3_6818 "/dev/ttySAC2"
    #define COM4_6818 "/dev/ttySAC3"

    //RK3568串口所对应的文件名
    #define COM0_3568 "/dev/ttyS0"
    #define COM1_3568 "/dev/ttyS1"
    #define COM3_3568 "/dev/ttyS3"
    #define COM4_3568 "/dev/ttyS4"
2.初始化串口
3.收发数据
    write
    read
4.关闭文件
```

2.gy39

有两种工作模式

```
1.只获取光照强度
2.只获取温湿度大气压强海拔
```

由外部控制器发送至 GY-39 模块（十六进制）

所有串口指令格式，帧头：0xa5

指令格式：帧头+指令+校验和(8bit) 各占一个字节 共三个字节

指令：

10000001=>0x81 获取光照强度

10000010=>0x82 获取温湿度大气压强海拔

校验和(前面的数据累加的和，只会保留低8位)

0xa5+0x81 = 0x26

0xa5+0x82 = 0x27

char cmd[3] = {0xa5,0x81,0x26};

write(fd,cmd,3);

char cmd1[3] = {0xa5,0x82,0x27};

如果是第一种模式：

```
接收数据，接收9个字节
char buf1[9] = {0};
read(fd,buf1,9);

buf[0] = 0x5a
buf[1] = 0x5a
buf[2] = 0x15

光照：
    Lux=(buf[4]<<24) | (buf[5]<<16) | (buf[6]<<8) | buf[7];
    Lux=Lux/100; (lux)
```

如果是第二种模式：

```
接收数据，接收15个字节
char buf2[15] = {0};
read(fd,buf2,15);
buf2[0] = 0x5a
buf2[1] = 0x5a
buf2[2] = 0x45

温度：
    T=(buf2[4]<<8)|buf2[5];
    T=T/100 单位℃
气压：Byte6~Byte9
     P=(buf2[6]<<24) | (buf2[7]<<16) | (buf2[8]<<8) | buf2[9]
     P=P/100 单位 pa
湿度：Byte10~Byte11
     Hum=(buf2[10]<<8)|buf2[11]
     Hum=Hum/100 百分制
海拔：Byte12~Byte13
    H=(buf2[12]<<8)|buf2[13] 单位 m
```

```
void gy39()
{
    //打开串口

    //初始串口

    //获取光照强度
    int res = write(fd,cmd,3);
    if(res != 3)
```

```
    {
        return ;
    }

    char buf1[9] = {0};
    read(fd,buf1,9);
    if(buf1[0]==0x5a&&buf1[1]==0x5a&&buf1[2]==0x15)
    {
        Lux=(buf1[4]<<24) | (buf1[5]<<16) | (buf1[6]<<8) | buf1[7];
        Lux=Lux/100;
    }

    //切换模式
    int res = write(fd,cmd1,3);
    if(res != 3)
    {
        return ;
    }
    char buf2[15] = {0};
    read(fd,buf2,15);
    if(buf2[0]==0x5a&&buf2[1]==0x5a&&buf2[2]==0x45)
    {
        //温度
        T=(buf2[4]<<8)|buf2[5];
        T=T/100;
        //湿度 大气压强海拔 自己写代码
    }
}
```

参考代码:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <termios.h>
#include <stdlib.h>
#include <netdb.h>
#include <string.h>
#include <errno.h>
//6818串口所对应的文件名
#define COM2 "/dev/ttySAC1"
#define COM3 "/dev/ttySAC2"
#define COM4 "/dev/ttySAC3"


int data[5] = {0};

//定义命令--->设置模块的工作方式
unsigned char cmd1[3]={0xa5,0x81,0x26};//设置连续光照模式
unsigned char cmd2[3]={0xa5,0x82,0x27};//设置温湿度气压海拔模式
unsigned char cmd3[3]={0xa5,0x83,0x28};//设置光照温湿度气压海拔模式
/* 设置串口参数（初始化串口） */
void init_tty(int fd)
{
    //1.先有保存串口属性的结构体
    //声明设置串口的结构体
    struct termios termios_new;
```

```
    //先清空该结构体
    bzero(&termios_new, sizeof(termios_new));

    //2.激活选项
    //cfmakeraw()设置终端属性，就是设置termios结构中的各个参数。
    cfmakeraw(&termios_new);
    //CLOCAL和CREAD分别用于本地连接和接受使能，因此，首先要通过位掩码的方式激活这两个选项。

    termios_new.c_cflag |= CLOCAL | CREAD;

    //3.设置波特率   一般地，用户需将终端的输入和输出波特率设置成一样的。
    //termios_new.c_cflag=(B9600);
    cfsetispeed(&termios_new, B9600);
    cfsetospeed(&termios_new, B9600);

    //4.设置字符大小,通过掩码设置数据位为8位
    //与设置波特率不同，设置字符大小并没有现成可用的函数，需要用位掩码。一般首先去除数据位中的位掩码，再重新按要求设置
    termios_new.c_cflag &= ~CSIZE;/* 用数据位掩码清空数据位设置 */
    termios_new.c_cflag |= CS8;

    //5.设置无奇偶校验
    termios_new.c_cflag &= ~PARENB;

    //6.设置一位停止位
    termios_new.c_cflag &= ~CSTOPB;

    //7.设置最少字符和等待时间
    //在对接收字符和等待时间没有特别要求的情况下，可以将其设置为0，则在任何情况下read()函数立即返回
    termios_new.c_cc[VTIME] = 10;
    termios_new.c_cc[VMIN] = 1;

    //8.用于清空输入/输出缓冲区
    tcflush (fd, TCIFLUSH);//对接收到而未被读取的数据进行清空处理

    //9.完成配置后，可以使用以下函数激活串口设置
    if(tcsetattr(fd,TCSANOW,&termios_new))//配置的修改立即生效
        printf("Setting the serial1 failed!\n");

}

//GY39模块
void GY39()
{
    //1.打开串口
    int ttyfd = open(COM2,O_RDWR);
    if(-1 == ttyfd)
    {
        perror("open ttyfd fail");
        return ;
    }
    //2.初始化串口
    init_tty(ttyfd);

    unsigned char rbuf1[9];
    unsigned char rbuf2[15];
    int Lux,T,P,HUM,H;
```

```
    int ret = write(ttyfd,cmd1,3);//获取光照强度
    if(ret!=3)
    {
        sleep(1);//等待完全写入
    }

    read(ttyfd,rbuf1,9);
    if(rbuf1[0]==0x5A&&rbuf1[1]==0x5A&&rbuf1[2]==0x15)//判断帧是否正常
    {
        Lux=(rbuf1[4]<<24)|(rbuf1[5]<<16)|(rbuf1[6]<<8)|rbuf1[7];
        data[0] = Lux=Lux/100;//最后一位不要 不显示小数
        printf("light = %d\n",Lux);
    }
    //获取气压海拔等....
    //S1 切换模式
    ret = write(ttyfd,cmd2,3);
    if(ret!=3)
    {
        sleep(1);//等待完全写入
    }

    read(ttyfd,rbuf2,15);
    if(rbuf2[0]==0x5A&&rbuf2[1]==0x5A&&rbuf2[2]==0x45)//判断帧是否正常
    {
        //温度
        T=(rbuf2[4]<<8)|rbuf2[5];
        data[1] = T=T/100;
        printf("T=%d\n",T);
        //气压湿度海拔自己完成 <---练习
        //气压
        P = (rbuf2[6]<<24)|(rbuf2[7]<<16)|(rbuf2[8]<<8)|rbuf2[9];
        data[2] = P = P/100;
        printf("P=%d\n",P);
        //湿度
        HUM=(rbuf2[10]<<8)|rbuf2[11];
        data[3] = HUM=HUM/100;
        printf("HUM=%d\n",HUM);
        //海拔
        H=(rbuf2[12]<<8)|rbuf2[13];
        data[4] = H=H;
        printf("H=%d\n",H);
    }
    close(ttyfd);
    return ;
}

int main()
{
    while(1)
    {
        GY39();
    }
}
```

也可以使用模式3，同时读取光照强度，温度，湿度....

```
void GY39_2()
```

```
{
    //1.打开串口
    int ttyfd = open(COM2,O_RDWR);
    if(-1 == ttyfd)
    {
        perror("open ttyfd fail");
        return ;
    }
    //2.初始化串口
    init_tty(ttyfd);

    unsigned char rbuf1[15];
    int Lux,T,P,HUM,H;

    //获取
    //切换模式3 获取光照气压海拔等....
    ret = write(ttyfd,cmd2,3);
    if(ret!=3)
    {
        sleep(1);//等待完全写入
    }

    read(ttyfd,rbuf2,3);//先读取3个字节
    if(rbuf1[0]==0x5A&&rbuf1[1]==0x5A&&rbuf1[2]==0x15)//光照
    {
        read(ttyfd,rbuf2+3,9-3);//再读取6个字节

        Lux=(rbuf1[4]<<24)|(rbuf1[5]<<16)|(rbuf1[6]<<8)|rbuf1[7];
        data[0] = Lux=Lux/100;//最后一位不要 不显示小数
        printf("light = %d\n",Lux);

    }
    else if(rbuf2[0]==0x5A&&rbuf2[1]==0x5A&&rbuf2[2]==0x45)//大气海拔温度湿度等...
    {
        read(ttyfd,rbuf2+3,15-3);//再读取12个字节
        //温度
        T=(rbuf2[4]<<8)|rbuf2[5];
        data[1] = T=T/100;
        printf("T=%d\n",T);
        //气压湿度海拔自己完成 <---练习
        //气压
        P = (rbuf2[6]<<24)|(rbuf2[7]<<16)|(rbuf2[8]<<8)|rbuf2[9];
        data[2] = P = P/100;
        printf("P=%d\n",P);
        //湿度
        HUM=(rbuf2[10]<<8)|rbuf2[11];
        data[3] = HUM=HUM/100;
        printf("HUM=%d\n",HUM);
        //海拔
        H=(rbuf2[12]<<8)|rbuf2[13];
        data[4] = H=H;
        printf("H=%d\n",H);
    }
    close(ttyfd);
    return ;
}
```

请根据参考代码，获取gy39模块数据，需要实现功能，点击传感器的按键，点击进入到下列的界面，进行数据的显示,当前页面需要实现功能：

1.刷新按钮，点击刷新，能够更新数据(即再调用gy39函数)

2.退出按钮，能够回到点餐界面

3.mq2烟雾模块

通信协议

1	0x86	读传感器的浓度值							
发送	0	1	2	3	4	5	6	7	8
	起始位	地址	命令	--	--	--	--	--	校验值
	0xFF	0x01	0x86	0	0	0	0	0	0x79
EXP.	FF 01 86 00 00 00 00 00 79								
返回	0	1	2	3	4	5	6	7	8
	起始位	命令	传感器浓度值		--	--	--	--	校验值
	0xFF	0x86	高字节	低字节	0	0	0	0	0xF5
EXP.	FF 86 00 85 00 00 00 00 F5								

```
unsigned char cmd[9] = {0xFF,0x01,0x86,0x00,0x00,0x00,0x00,0x00,0x79}; //传感器采集命令 用来发送数据
unsigned int concen = 0;//保存获取到的烟雾浓度的值
```

参考代码：

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <termios.h>
#include <stdlib.h>
#include <netdb.h>
#include <string.h>
#include <errno.h>
//6818串口所对应的文件名
#define COM2_6818 "/dev/ttySAC1"
#define COM3_6818 "/dev/ttySAC2"
#define COM4_6818 "/dev/ttySAC3"
```

```c
//RK3568串口所对应的文件名
#define COM0_3568 "/dev/ttyS0"
#define COM1_3568 "/dev/ttyS1"
#define COM3_3568 "/dev/ttyS3"
#define COM4_3568 "/dev/ttyS4"

unsigned char cmd[9] = {0xFF,0x01,0x86,0x00,0x00,0x00,0x00,0x00,0x79}; //传感器采集命令 用来发送数据
unsigned int concen = 0;//保存获取到的烟雾浓度的值

/* 设置串口参数（初始化串口） */
void init_tty(int fd)
{
    //1.先有保存串口属性的结构体
    //声明设置串口的结构体
    struct termios termios_new;
    //先清空该结构体
    bzero(&termios_new, sizeof(termios_new));

    //2.激活选项
    //cfmakeraw()设置终端属性，就是设置termios结构中的各个参数。
    cfmakeraw(&termios_new);
    //CLOCAL和CREAD分别用于本地连接和接受使能，因此，首先要通过位掩码的方式激活这两个选项。

    termios_new.c_cflag |= CLOCAL | CREAD;

    //3.设置波特率  一般地，用户需将终端的输入和输出波特率设置成一样的。
    //termios_new.c_cflag=(B9600);
    cfsetispeed(&termios_new, B9600);
    cfsetospeed(&termios_new, B9600);

    //4.设置字符大小,通过掩码设置数据位为8位
    //与设置波特率不同，设置字符大小并没有现成可用的函数，需要用位掩码。一般首先去除数据位中的位掩码，再重新按要求设置
    termios_new.c_cflag &= ~CSIZE;/* 用数据位掩码清空数据位设置 */
    termios_new.c_cflag |= CS8;

    //5.设置无奇偶校验
    termios_new.c_cflag &= ~PARENB;

    //6.设置一位停止位
    termios_new.c_cflag &= ~CSTOPB;

    //7.设置最少字符和等待时间
    //在对接收字符和等待时间没有特别要求的情况下，可以将其设置为0，则在任何情况下read()函数立即返回
    termios_new.c_cc[VTIME] = 10;
    termios_new.c_cc[VMIN] = 1;

    //8.用于清空输入/输出缓冲区
    tcflush (fd, TCIFLUSH);//对接收到而未被读取的数据进行清空处理

    //9.完成配置后，可以使用以下函数激活串口设置
    if(tcsetattr(fd,TCSANOW,&termios_new))//配置的修改立即生效
        printf("Setting the serial1 failed!\n");

}
```

```c
void smoke()
{
    //打开串口
    int fd = open(COM1_3568,O_RDWR);
    if(fd == -1)
    {
        perror("failed open");
        return ;
    }
    //初始串口
    init_tty(fd);

    while(1)
    {
        //发送获取烟雾浓度命令
        int res = write(fd,cmd,9);
        if(res != 9)
        {
            return ;
        }
        char buf1[9] = {0};
        read(fd,buf1,9);//用来保存接收的数据
        if(buf1[0]==0xff&&buf1[1]==0x86)
        {
            concen = buf1[2]<< 8 | buf1[3];
        }
        printf("烟雾浓度为%d\n",concen);
    }
    close(fd);

}

int main()
{
    smoke();
}
```